立项答辩 | Agent 安全 | Graphviz 图解

面向 AI Agent 的运行时安全观测与控制平台

先说明网上 Agent 安全的现状与未来，再明确本项目定位：不是再造一个 Agent，而是为 Agent 执行链提供可验证、可回放、可管控的安全底座。

eBPF
内核事实层

Hooks
语义声明层

Policy
运行时控制层

定位一句话

把 Agent 的「承诺」和操作系统里的「事实」对齐起来

目录 / 多章节答辩结构

用 4 个章节把“现状—定位—路线—实施”讲清楚，流程图由 Graphviz 生成并插入。

CH1 **现状与趋势** 为什么 Agent 安全已经从内容问题变成执行链问题

CH2 **项目定位** 本项目做什么、不做什么，以及与现有方案的边界

CH3 **技术路线** 事实层、语义层、控制层、展示层如何闭环

CH4 **实施与验证** 阶段计划、评测方法、风险边界和预期成果

CHAPTER 1

现状与趋势

先回答为什么现在需要做 Agent 运行时安全。

现状

威胁链

未来治理

01

多章节答辩结构

网上 Agent 安全现状：风险从“回答错”变成“执行错”

Agent 拥有工具、文件、网络、浏览器和代码执行权限后，攻击面变成连续运行时链路。

现状 1：自主性扩大

- LLM 应用风险已包含 Excessive Agency：过度授权会带来可靠性、隐私与信任问题。
- Agent 的“多步计划 + 工具调用”让单点过滤难以覆盖完整后果。

现状 2：攻击入口外移

- 提示注入可以藏在网页、Issue、README、依赖脚本和本地文件中。
- 攻击者不一定要突破系统，只要诱导 Agent 合法地做错事。

现状 3：可观测性不足

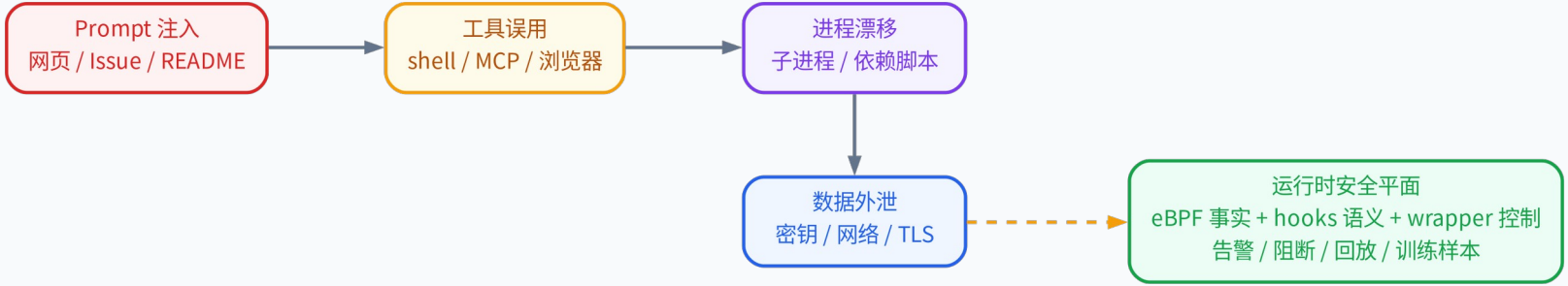
- 企业通常能看到最终文件变更，却难还原每个进程、网络端点和工具调用。
- 缺少运行时事实会导致误报、漏报和事故复盘困难。

答辩引导：这不是“模型安全”单点问题，而是 Agent 执行链安全问题。

Sources: OWASP Top 10 for LLM Applications / OWASP Agentic AI Threats and Mitigations.

典型威胁链： Prompt → Tool → Process → Data

Graphviz 流程图：把网上讨论的 Agent 安全问题翻译成可答辩的工程攻击面。



攻击特点

- 跨层联动：自然语言、工具、进程、网络同时参与。

防御难点

- 只拦截易误伤，只看日志又太晚。

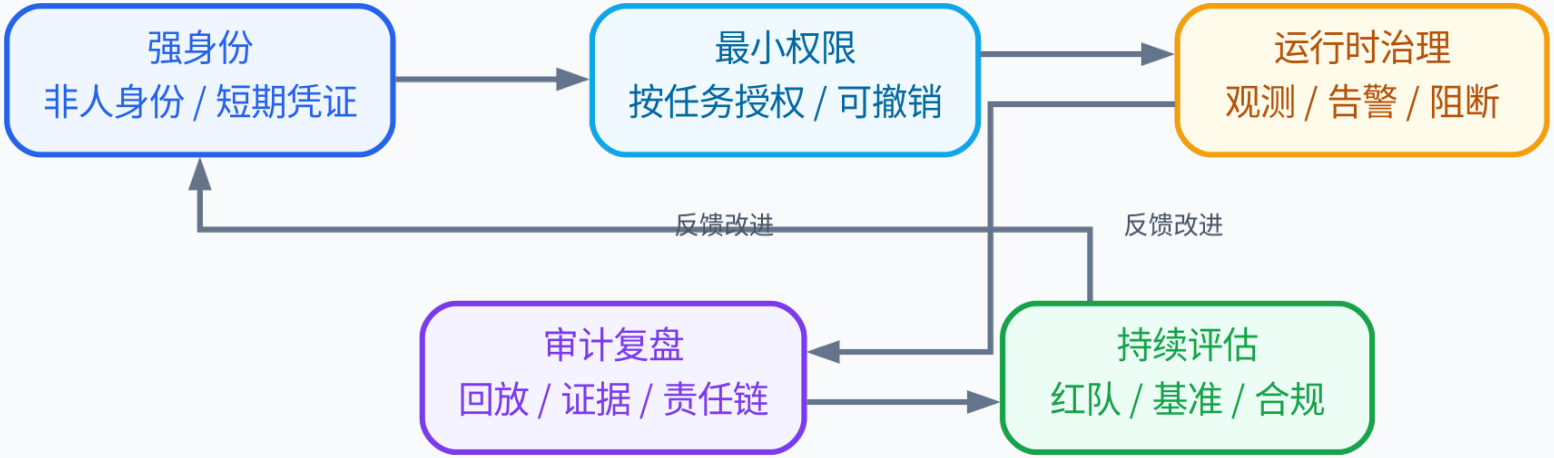
项目切入口

- eBPF 事实 + hooks 语义 + wrapper 控制。

关键判断： Agent 攻击不是单点漏洞，而是“提示注入 → 工具误用 → 进程漂移 → 数据外泄”的执行链问题。

未来趋势： Agent 安全会走向“可控自治”

Graphviz 闭环图：未来治理重点从提示词安全扩展为身份、权限、运行期治理、审计和持续评估。



结论：未来不是简单“禁用 Agent”，而是让 Agent 在受约束、可解释、可追责的环境里工作。

Sources: Five Eyes “Careful Adoption of Agentic AI Services” (2026-05-01); NIST AI RMF / GenAI Profile.

CHAPTER 2

项目定位

再回答本项目做什么、不做什么、和别人有什么区别。

定位　边界　目标

02

多章节答辩结构

项目定位： Agent 执行链安全的“黑匣子 + 刹车系统”

Graphviz 定位图：本项目不做大模型本体，也不做完整沙箱；聚焦事实采集、语义关联和策略控制。

一句话定位：给开发者和安全团队一个低侵入、可复盘、能干预的 Agent 运行时安全平面。

05

立项目标与研究内容

把定位落成可验收的任务，而不是停留在“安全愿景”。

目标 1	**可观测**	构建 Agent 执行链事件模型：进程、文件、网络、 TLS 、工具调用、策略结果。
目标 2	**可解释**	关联“ Agent 声称的任务意图”和“系统实际行为”，识别语义不一致。
目标 3	**可控制**	通过 wrapper / hooks / runtime gating 输出 ALLOW 、 ALERT 、 BLOCK 、 REWRITE 。
目标 4	**可复盘**	提供前端图谱、 JSONL 回放、 OTLP/MCP 导出与训练样本回流。

CHAPTER 3

技术路线

把定位落到系统结构：事实层、语义层、控制层、展示层。

Graphviz

eBPF

策略闭环

03

多章节答辩结构

技术路线：事实层 + 语义层 + 控制层 + 展示层

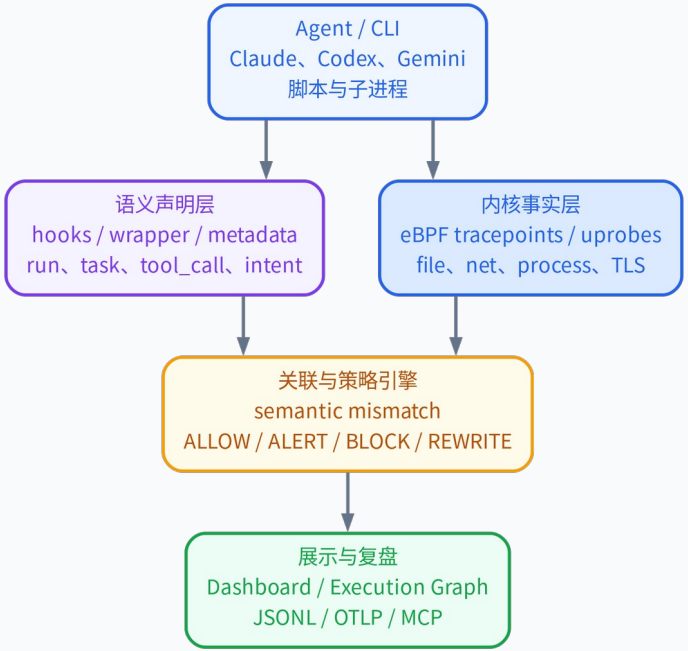
Graphviz 架构图：以 Linux 运行时事实为底座，再接入 AI CLI 语义和策略执行。

语义输入

• hooks / wrapper 描述 Agent 自称要做什么。

事实输入

• eBPF 记录进程、文件、网络和 TLS 事实。



输出能力

• 策略命中、图谱复盘、JSONL/OTLP/MCP 导出。

已有基础： Go/eBPF/Vue/wrapper/hooks 原型｜核心创新：语义声明 vs OS 事实一致性判定｜答辩亮点：事件流、执行图谱、策略命中和回放

系统原型能力：已经能支撑立项演示

从“论文式设想”推进到可以运行、可以截图、可以现场讲解的工程原型。

9+	4+	UDS	TLS	Graph	OTLP
核心 syscall tracepoints	AI CLI hook 适配	wrapper 本地控制	明文片段捕获与脱敏	执行拓扑回放	可观测性导出

观测面

- exec/open/connect/mkdir/unlink/ioctl/bind/send/recv 等事件。
- TLS request/response 片段脱敏、截断后展示。

控制面

- wrapper 规则返回 ALLOW/BLOCK/ALERT/REWRITE 。
- 高危功能默认 runtime-gated 。

复盘面

- Dashboard 、 Execution Graph 、 JSONL replay 、 MCP/OTLP 输出。
- 可转成训练样本继续优化。

08

创新点：不是“多加日志”，而是建立 Agent 安全语义闭环

答辩中应强调差异化：低侵入观测、语义事实对齐、策略闭环、可复盘证据。

1. 双层证据

• hook/wrapper 提供意图；eBPF 提供事实。
• 两者不一致时形成语义告警。

2. 低侵入

• 无需修改 Agent 模型。
• 通过 PID 注册、进程继承和系统调用观测接入。

3. 可干预

• 不是事后看日志；wrapper 能在命令执行前决策。
• 策略变更可在 UI 中管理。

4. 可积累

• 事件可回放、导出和转训练样本。
• 为后续 ML/LLM 风险评分提供数据基础。

答辩表达：项目的贡献是把 Agent 安全从“提示词防护”推进到“运行时证据与控制平面”。

CHAPTER 4

实施与验证

最后说明阶段计划、可行性、风险边界和预期成果。

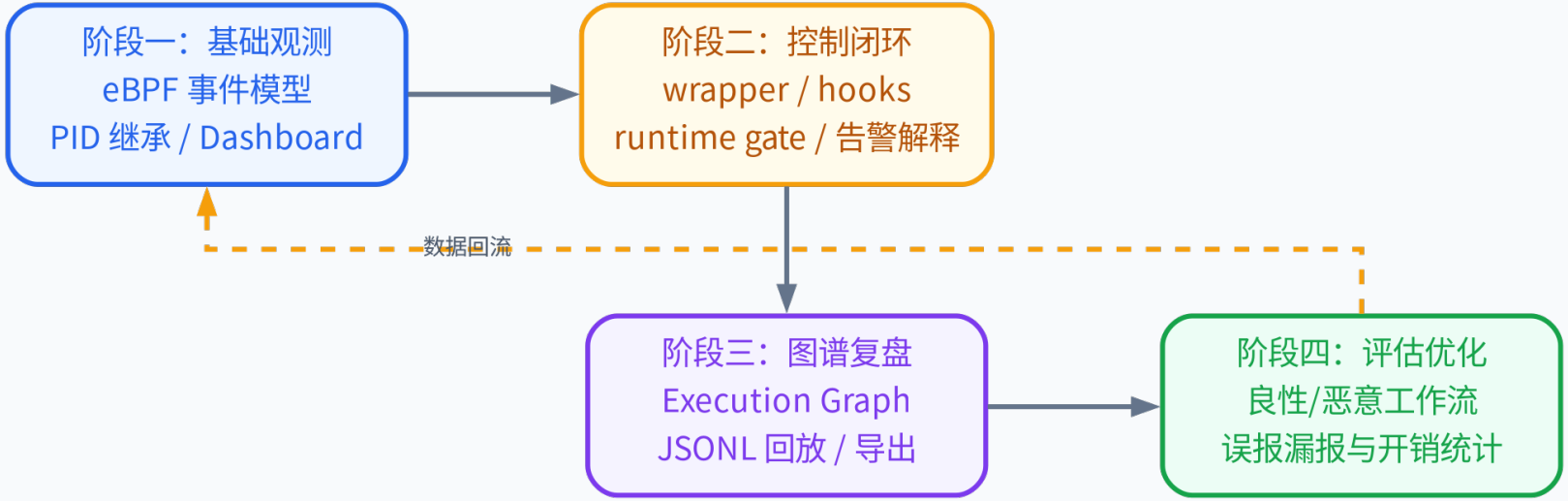
计划　验证　成果

04

多章节答辩结构

实施计划与阶段成果

Graphviz 路线图：按立项答辩习惯给出明确路线，强调阶段递进和评估回流。



预期成果：原型系统、威胁模型与安全模型文档、典型 Agent 工作流评测、演示视频 / 截图、可投稿技术报告。

10

可行性：仓库已有工程骨架，后续重点是收敛与验证

说明为什么该项目不是空想：核心链路已经跑通，立项后可以集中打磨边界、评测和展示。

工程基础

- Go + eBPF 后端已经具备 syscall ringbuf、pinned maps、WebSocket/API。
- Vue 3 前端已有 Dashboard、Network、Execution Graph、Config 等页面。

安全基础

- release-mode token、hook secret、UDS 0600、peer credential、危险功能 gate。
- 已有 threat-model/security-model/policy-semantics 文档。

验证基础

- 事件 JSONL、图谱回放、benchmark、ML/LLM 评分链路可作为评估材料。
- 后续重点补齐代表性 Agent 场景集。

风险控制：诚实说明非目标——不防 root/ 恶意内核，不替代容器或企业 EDR；用清晰边界换取可落地。

结论：项目定位清晰、问题真实、路线可落地

从网上 Agent 安全趋势出发，落到本项目可实现的运行时安全底座。

答辩收束三句话

1. Agent 安全的核心矛盾，是自主执行能力和可控性之间的矛盾。
2. 本项目的定位，是补齐 Agent 执行链的事实观测、语义关联和策略控制。
3. 现有原型已经覆盖关键链路，后续工作可以围绕评测、体验和硬化持续推进。

项目关键词

- 运行时证据
- 最小权限
- 语义一致性
- 可回放审计
- 策略闭环

谢谢！

主要来源： Five Eyes Careful Adoption of Agentic AI Services (2026-05-01); OWASP Agentic AI Threats and Mitigations (2025); OWASP Top 10 for LLM Applications; NIST AI RMF / GenAI Profile。